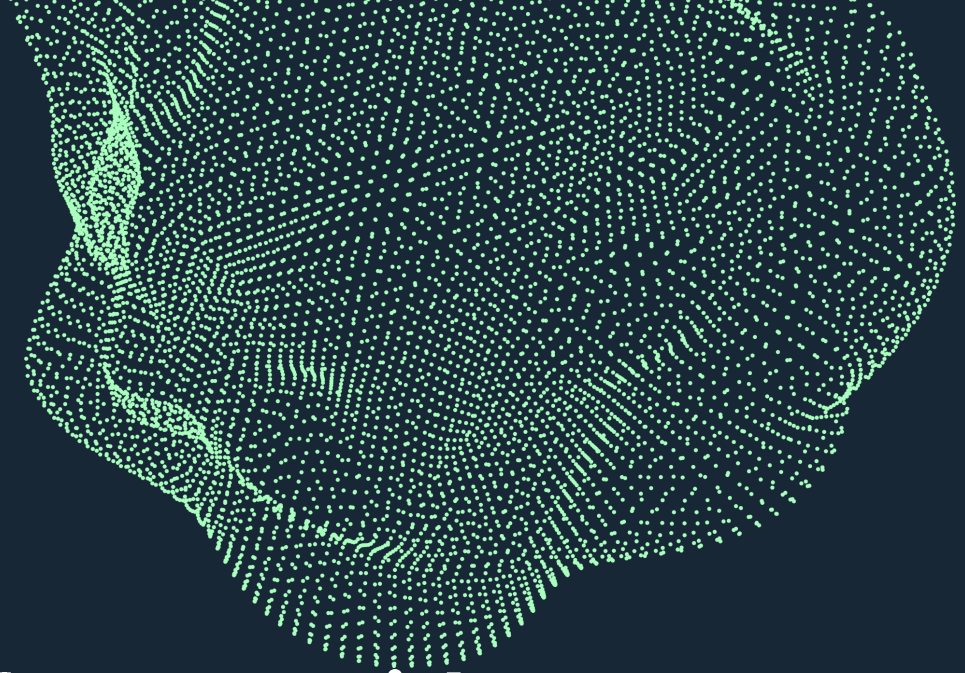


appliedAI Institute Masterclass

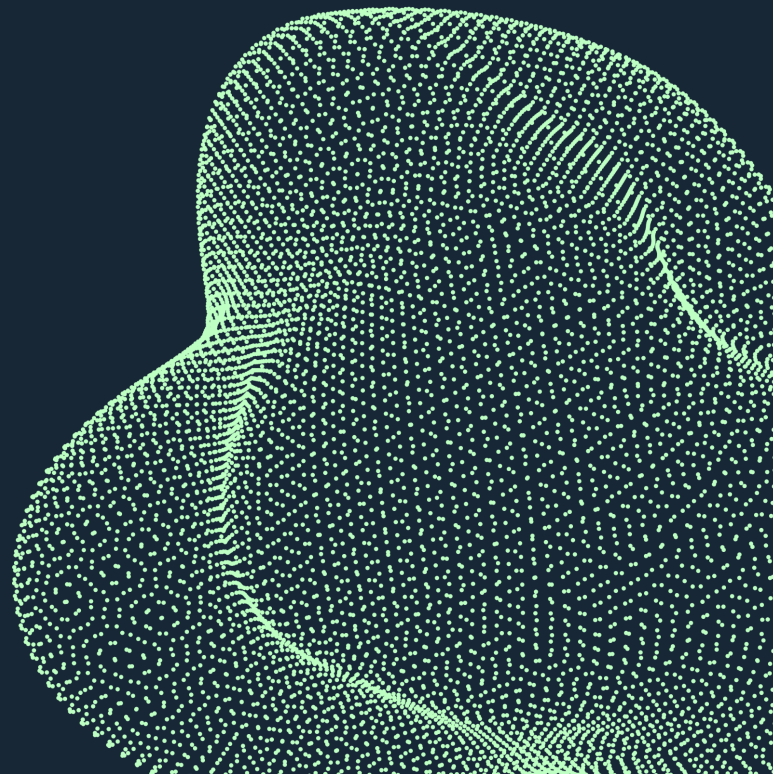
Building AI Agents with Pydantic AI

Dr. Jan Willem Kleinrouweler

A joint initiative

appliedAI institute for europe

Introduction

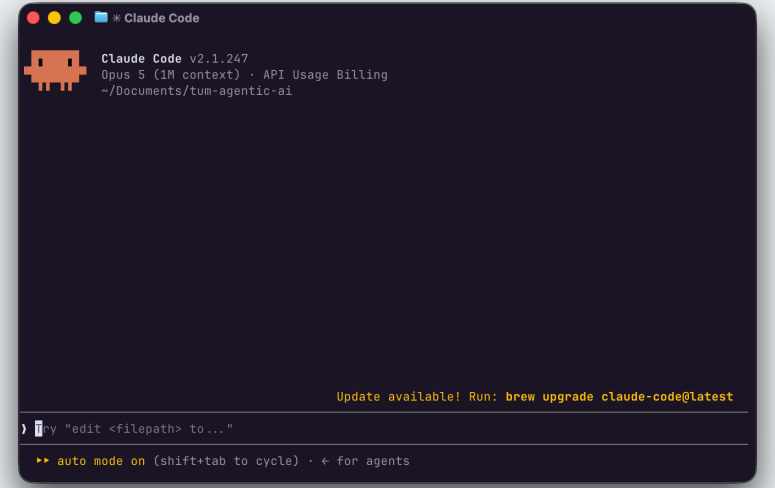


AI Agents Are Systems That Act

Most people interact with LLMs through chat interfaces. The same models, however, can be used to complete complex multi-step processes.

- Transition from **"AI that helps"** to **"AI that does"**.

- An LLM can interpret information, generate responses, and propose actions.
- Software around the LLM enables it to interact with external systems.



What Is an AI Agent?

There is no universally accepted definition of an AI agent, but we can identify characteristics that are shared by many agents systems:

- An agent is given a **clear task or goal**.
- An **LLM can decide what to do next** based on the available information.
- The agent can **interact with its environment** through tools.
- The agent can **adapt its actions** based on intermediate results.

Standard LLM Call

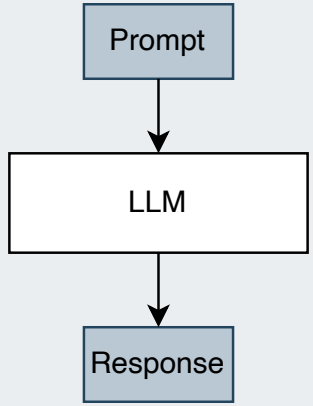
Baseline model

Pattern:

Prompt → Model → Response

Characteristics:

- Single request-response interaction
- All context must fit in the prompt
- Model cannot verify or act on external systems



Augmented LLM

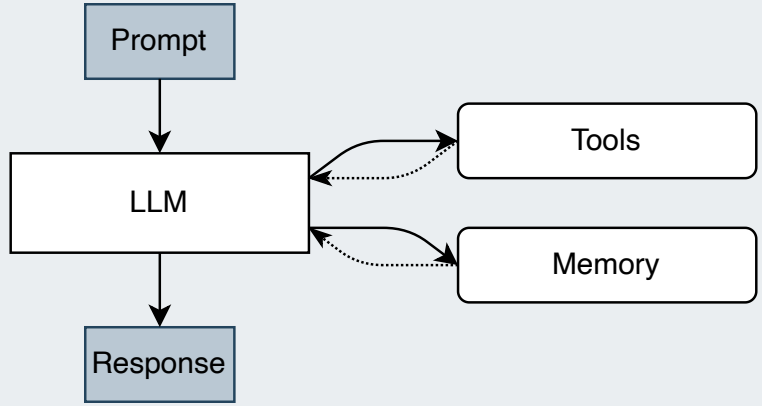
Capability-enhanced model

Pattern:

Prompt → Model ←→ Tools → Response

New capabilities:

- **Tool calling** (APIs, search, code execution, databases)
- **Memory** (conversation history, long-term state)
- Optional short internal tools loops before final answer

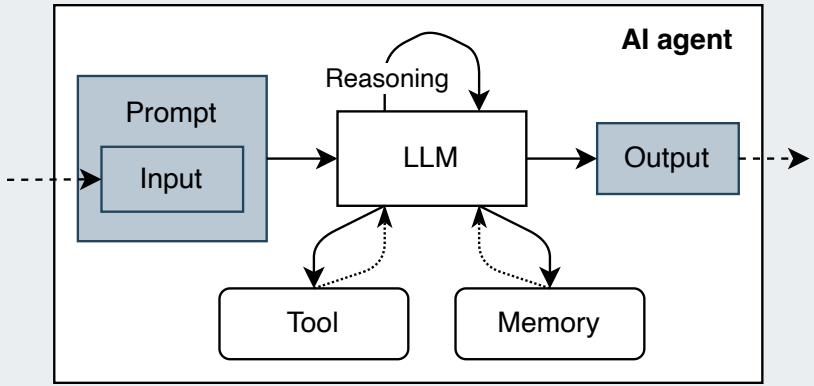


The Core Architecture

LLM + Harness

The LLM provides the decision making (i.e., the brain), while the harness provides the infrastructure and tooling needed to execute tasks (i.e., the body):

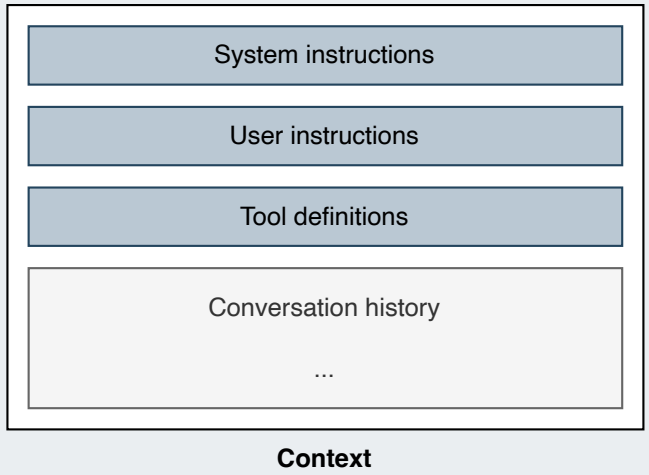
- **LLM**: Interprets information, generate responses, and select next actions.
- **Harness:** Constructs the model requests, processes the responses, executes tools, and maintains the conversation state.
  - **Instructions:** Define the behavior and the task.
  - **Tools:** Capabilities made available to the model through the harness.
  - **State:** Preserves the information across multiple interactions.



How the Harness Communicates with an LLM

The harness makes a request to the model containing all the information that is relevant for the model to make a decision.

- **System/developer instructions:** How the model should behave.
- **User instructions:** What task to perform.
- **Available tools:** Which capabilities the model may request.
- **Conversation and tool history:** What has happened so far.



LLM Response

The model generates a response based on the supplied context. The harness interprets the model's response and executes the next action:

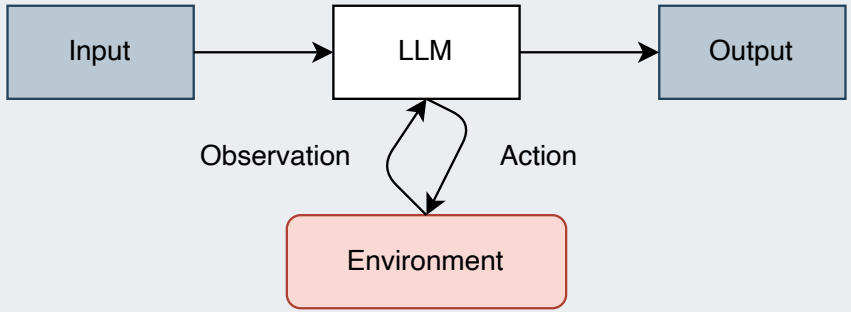
- **Text output:** An answer, explanation, or recommendation.
- **Structured output:** Data is a defined structure (e.g., JSON object).
- **Tool-call request:** Request to execute a specific function with arguments.
- **Reasoning:** Reasoning information or summaries (if exposed via the API).

The Agent Loop

Decide, Act, Observe, and Repeat

An agent can perform a multi-step task by repeatedly determining the next action and incorporating its results.

1. **Decide**
   Evaluate the current task and context.
   Decide whether is an action is necessary.
2. **Act**
   Request a tool call when required.
3. **Observe**
   Receive the tool result of the executed tool (or a denial).
4. **Repeat**
   Update the execution context and repeat



Prompts and Instructions

Behavior vs. Task

A reusable agent needs a stable definition of how it should behave, while individual tasks may change.

- **System/developer instructions:**
  - Responsibilities
  - Boundaries (what can the agent do, and what should the agent not do)
  - Policies
  - Instructions on how to use tools
  - Output expectations (e.g., structured output schema)
- **User instructions:**
  - Specific request and task definition
  - Relevant information to support the execution of the task.

Tools
What Can the Agent Actually Do?

Tools allow the agent to retrieve information or request changes to external systems:

- **Information retrieval:** Search and read documents, query databases.
- **Actions:** Create tickets, update database records, send notifications.

- Each tools has a **purpose, parameters, and expected result**.
- The model selects a tool and provides its arguments.
- The harness executes the tool and returns the results to the model.

Examples:

```
get_employee_info(employee_id)
execute_bash(command)
get_return_policy(product_category)
search_web(query, limit)
send_slack_message(channel_id, message)
```

State and Memory

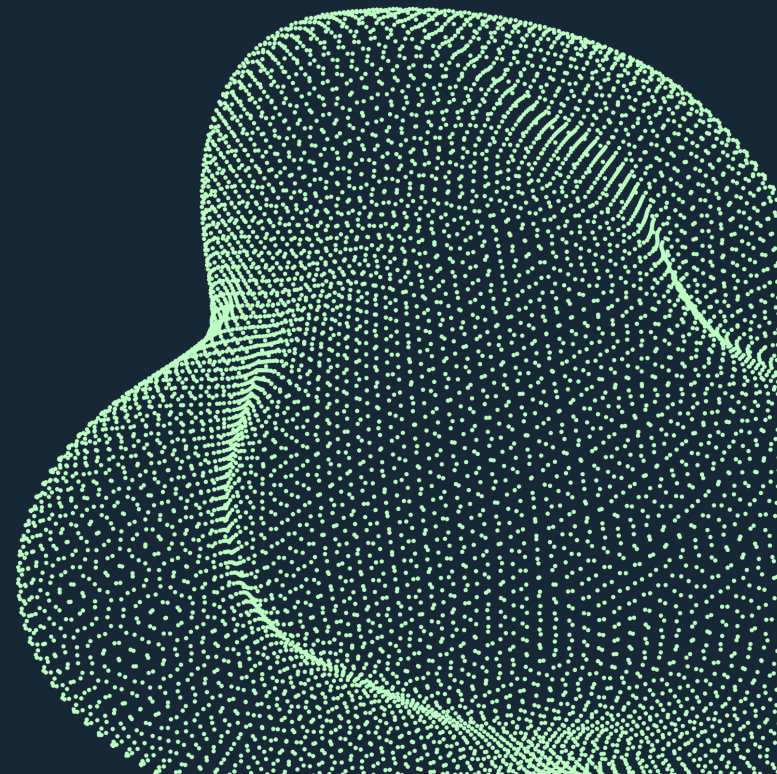
Individual model requests are stateless. The harness has to preserve and **manage the agent state**:

- Maintain information about the task currently being performed.
- Preserve the past messages and results across the agent loop.
- Keep track of intermediate results and actions (do not send the payment twice!)

Memory capabilities allow the agent to retrieve relevant information from earlier tasks:

- Conversation history
- Long-term memory is a capabilities that you can expose to your agent as tools

Quiz: Is this an agent?



Quiz
Is this an agent?

Retrieval agent:

An agent that retrieves as PDF document from a Web server and turns it into Markdown.

Quiz
Is this an agent?

Summarization agent:

An agent that receives a document and creates a bullet-point summary.

Quiz
Is this an agent?

Invoice extraction agent:

An agent that receives an invoice via an email, extracts the details, and updates the database.

Quiz
Is this an agent?

IT support agent

An agent that investigates an IT ticket, check the system status, determines the cause, and request actions to solve the IT ticket.

When Should You Build an Agent?

Before you start building your agent, examine which parts of the business problem require an agent and what can be implemented using conventional software.

- Use **deterministic software** when the procedure is fully known.
- Use a **standalone LLM call** for bounded generation tasks.
- Use a **workflow** when the execution sequence is predetermined.
- Use an **agent** for complex multi-step tasks.

Prefer the simplest architecture that satisfies the requirements. An AI system can mix & match any of these components!

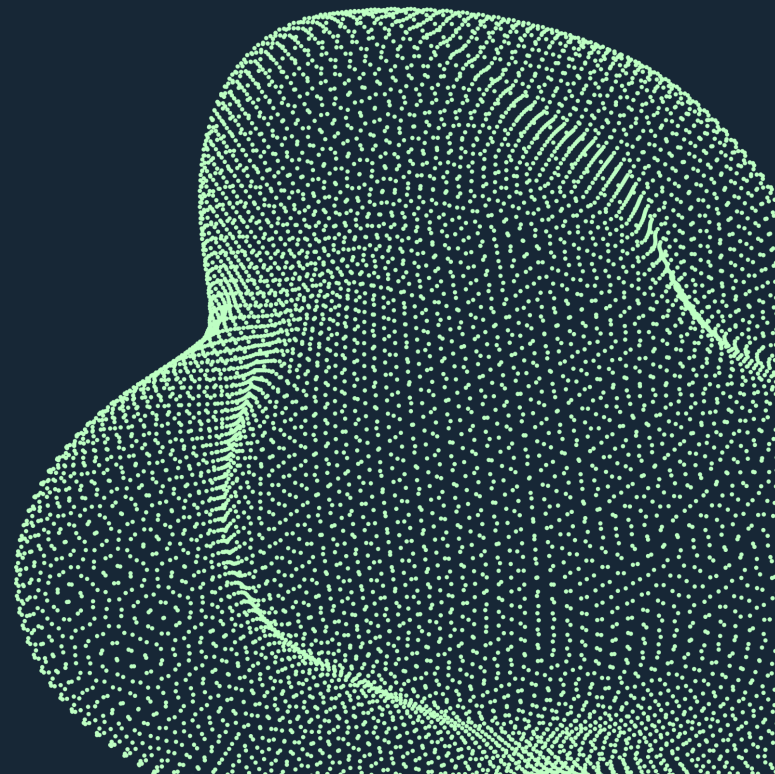
When Should You Build an Agent?

AI Agents are most justified when the task is goal-directed, multi-step, and the execution path is difficult to predefine in advance.

- **Open-ended execution**:
  - Unpredictable number of steps
  - Cannot be reliably implemented as fixed workflow
- **Essential tools use**:
  - Requires APIs, databases, code execution, etc.
  - Real-world interaction is part of the task
- **Feedback loops available**:
  - The system can validate progress using tools, tests, rules, and external state
- **Clear success criteria**:
  - Objective definition of done
- **Safe execution environment**:
  - Sandbox or execution harness
  - Human checkpoints and oversight
- **Sufficient business value**:
  - Justify latency, costs, and engineering complexity

appliedAI institute for europe

Building a coding assistant!



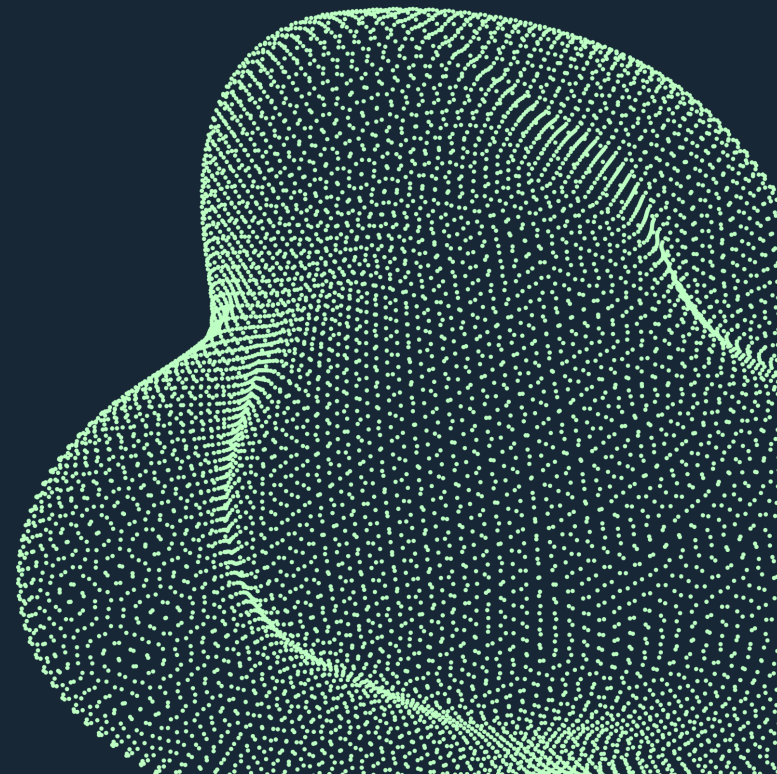
Building a coding assistant

A Pydantic AI agent that can perform simple coding tasks

appliedAI institute for europe

appliedAI institute for europe

Creating and running Pydantic AI Agents



Configure the model provider

Configuring a model provider in Pydantic AI defines how the agent connects to an LLM backend.

OpenAI & OpenAI-compatible APIs:

```
from pydantic_ai.providers.openai import OpenAIProvider

provider = OpenAIProvider(
    base_url = "https://api.openai.com/v1",
    api_key = "sk-..."
)
```

Anthropic API:

```
from pydantic_ai.providers.anthropic import (
    AnthropicProvider
)

provider = AnthropicProvider(
    base_url="https://api.anthropic.com",
    api_key="sk-ant-...",
)
```

Configure the model

Once the provider is configured, you bind a specific model to it.

OpenAI API:

```python
from pydantic_ai.models.openai import (
    OpenAIResponsesModel
)

model = OpenAIResponsesModel(
    model_name="gpt-5.1-codex-mini",
    provider=provider,
)
```

Anthropic API:

```python
from pydantic_ai.models.anthropic import (
    AnthropicModel
)

model = AnthropicModel(
    model_name="claude-sonnet-4-5",
    provider=provider,
)
```

Configure the model

Most open-weight models- and gateway providers use the OpenAI Chat Completions API.

OpenAI chat completions API:

```
from pydantic_ai.models.openai import (
    OpenAIChatModel
)

model = OpenAIChatModel(
    model_name="gpt-5.1-codex-mini",
    provider=provider,
)
```

Create the agent

With the model configured, you can instantiate an agent by pairing it with task-specific instructions.

```
from pydantic_ai import Agent

agent = Agent(
    model=model,
    instructions="You are a Python coding agent. ...",
)
```

Running the agent with `run()`

appliedAI institute for europe

`run()` executes the agent to completion and returns an **`AgentRunResult`**. Use it when you want the final (validated) output and run metadata in one object.

```python
result = await agent.run("Write a function to parse a CSV file.")

print(result.output)          # final output
print(result.usage())         # token / request usage
print(result.new_messages())  # messages from this run
print(result.all_messages())  # full conversation history
```

Run usage

The **`usage()`** method of the **`AgentRunResult`** provides information about the usage of the full agent run:

- Number of requests
- Number of tool calls
- Token usage

```
RunUsage(
    input_tokens=6515,
    cache_read_tokens=6240,
    output_tokens=1290,
    details={
        'reasoning_tokens': 260,
        'image_tokens': 0,
    },
    requests=5,
    tool_calls=4,
)
```

`ModelMessage` structure

Agent runs exchange **`ModelMessage`** objects with the model. You can retrieve the full message history with the **`all_messages()`** method.

The message history contains two types of messages:

`ModelRequest` - message send to the model:

- System prompt
- User prompt
- Tool returns

`ModelRespsonse` - message received from the model:

- Text
- Thinking (reasoning text)
- Tool call requests

`ModelRequest` message

Example: Explain the CSV parser

The first message in the exchange is a **`ModelRequest`** message.

```
ModelRequest(
    parts=[
        UserPromptPart(
            content='Explain the csv parser',
            timestamp=datetime.datetime(...)
        ],
    timestamp=datetime.datetime(...),
    instructions="You are a Python coding agent.\n ...'",
    run_id='019d85ec-7fff-7548-b4bd-f20a5627b0fd',
)
```

`ModelResponse` message

Example: Explain the CSV parser

The model replies to a **`ModelRequest`** message with a **`ModelResponse`**:

```
ModelResponse(
    parts=[
        ThinkingPart(content="The user is asking me to explain the CSV parser...", id='reasoning'),
        TextPart(content="I'll explore the project structure to find the CSV parser implementation."),
        ToolCallPart(
            tool_name='search_files',
            args='{"pattern": "**/*.py"}',
            tool_call_id='call_2b5b'
        ),
    ]
    model_name='z-ai/glm-5',
    timestamp=datetime.datetime(...),
    provider_url='https://api.***/v1',
    finish_reason='tool_call',
    run_id='019d85ec-7fff-7548-b4bd-f20a5627b0fd',
)
```

`ModelRequest` message

Example: Explain the CSV parser

Pydantic AI feeds back the tool call result by making another request to the model.

```
ModelRequest(
    parts=[
        ToolReturnPart(
            tool_name='search_files',
            content=['csv_parser.py'],
            tool_call_id='call_2b5b',
            timestamp=datetime.datetime(...)
        ),
    ],
    timestamp=datetime.datetime(...),
    instructions="You are a Python coding agent.\n ...",
    run_id='019d85ec-7fff-7548-b4bd-f20a5627b0fd'),
)
```

Running the agent with `run_stream()`

`run_stream()` returns a **`StreamedRunResult`**. Use **`stream_text()`** for incremental text output.

```
async with agent.run_stream("Explain how CSV parsing works") as result:
    async for chunk in result.stream_text():
        print(chunk, end="")
```

Running the agent with `iter()`

`iter()` exposes the agent graph step by step. It is useful for debugging, observability, custom control flows, and inspecting tool executions as it happens.

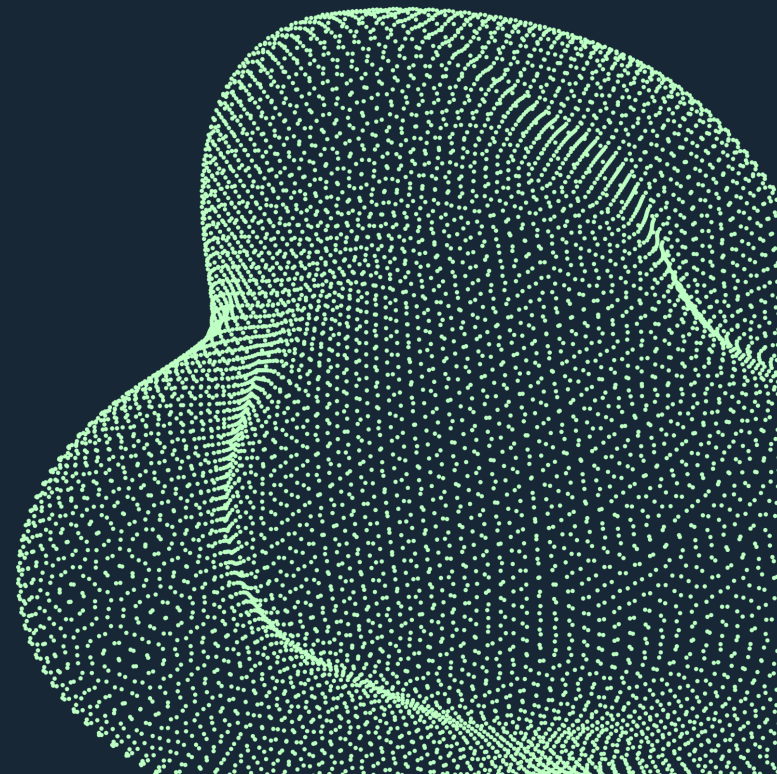
```
async with agent.iter("Write a function ...") as run:
    async for node in run:
        print(node)
```

- **`UserPromptNode`**: captures the user instructions
- **`ModelRequestNode`**: represents the request that is sent to the model
- **`CallToolsNode`**: processes the model response and executes tool calls
- **`End`**: terminates the graph and contains the final result

appliedAI institute for europe

Exercise 🎒

Your first LLM call



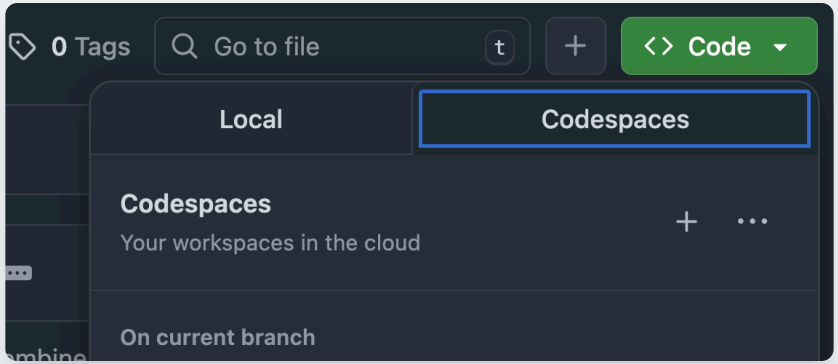
Exercise 1: Your first LLM call

Environment setup

The easiest way to work on the exercises is to use a **GitHub codespace**:

1. Sign in to your GitHub account
2. Navigate to the `aai-institute/agentic-ai-masterclass` repository
3. Start a new codespace in the cloud
4. Initialize the Python environment:

```
$ uv sync
$ uv run coding-assistant
```



Exercise 1: Your first LLM call

Goal: Establish a minimal working setup and make your first call to a language model.

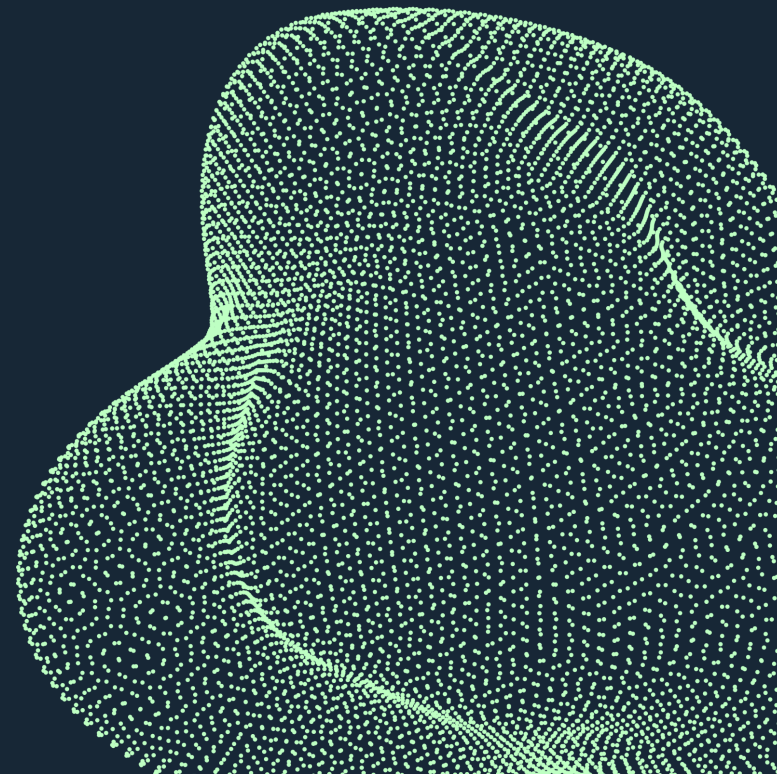
1. Configure an OpenAI model provider

2. Configure the model

3. Create and run the agent

appliedAI institute for europe

Agent memory and conversation state



Message history

Agents runs are **stateless by default***. This means calling **`run()`** for a second time won't include any past conversation history, unless it is explicitly provided.

```
result = await agent.run("Create a plan for implementing a CVS file parser.")
message_history = result.all_messages()

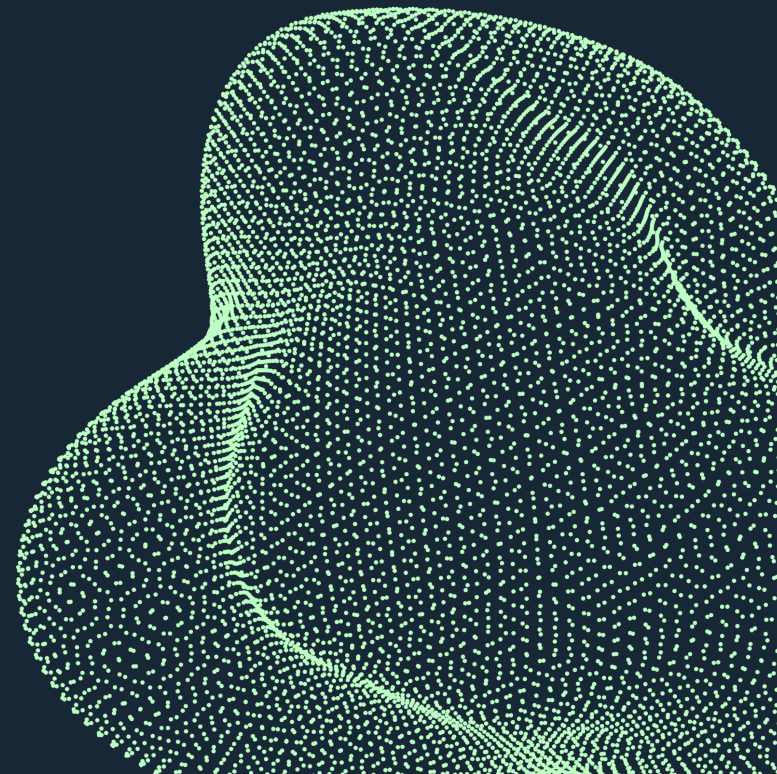
result = await agent.run("Great, implement it!", message_history=message_history)
```

* The LLM calls inside `run()` are also stateless, but Pydantic AI is handling the conversation state for us.

appliedAI institute for europe

Exercise 🎒

Maintaining conversation state



Exercise 2: Maintaining conversation state

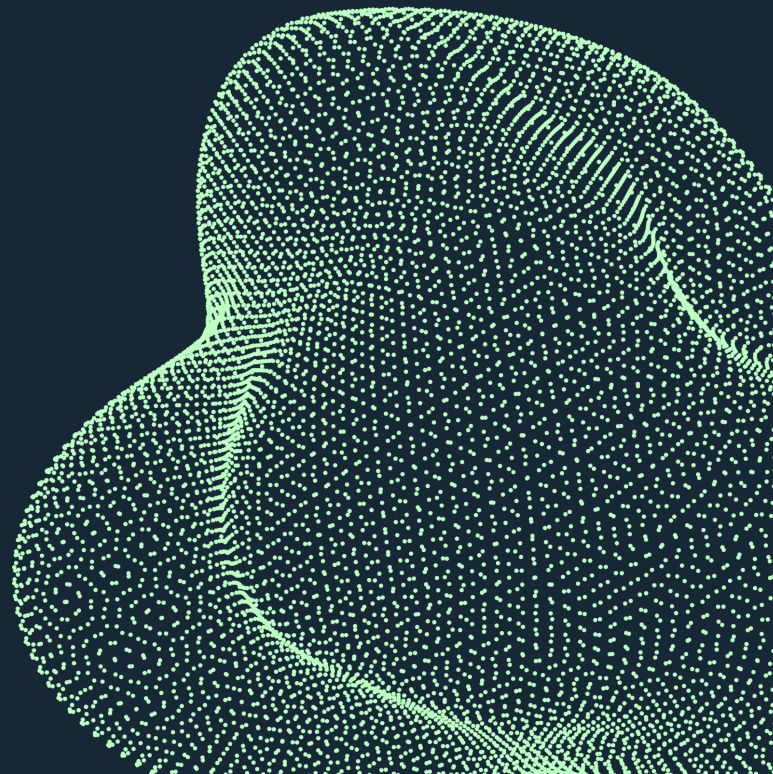
Goal: Extend your agent to become a conversational agent.

1. Wrap the agent `run()` call in a `while True` loop

2. Capture the message history with `all_messages()`

3. Pass the `message_history` in the `run()` method

Tools



Function tools

Function tools are normal Python functions that take arguments provided by the model, and the return values are send back to the model.

```python
def add(a: float, b: float) -> float:
    """Add a to b."""
    return a + b

def sub(a: float, b: float) -> float:
    """Subtract b from a."""
    return a - b
```

```python
from pydantic_ai import Agent

agent = Agent(
    model=model,
    instructions="You are a math wizard.",
    tools=[add, sub],
)
```

Function specification
(OpenAI tool definitions)

The function definitions are declared in the tools parameter of each API request.

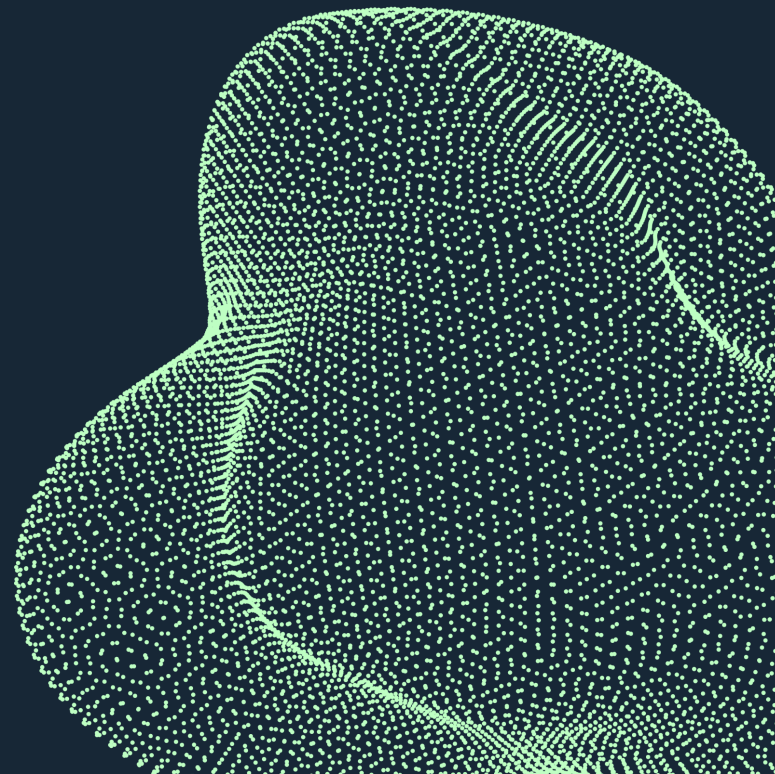
- The tool and parameter names come from the **Python function definition**.
- The descriptions are extracted from the **function docstring**.

Parameters must be representable as JSON objects:

- Use **JSON data types** (string, number, boolean, object, array)
- A **single parameter** that can be **represented as JSON object**: dataclass, Pydantic model.

```
{
    "type": "function",
    "name": "add",
    "description": "Add a to b.",
    "parameters": {
        "type": "object",
        "properties": {
            "a": {
                "type": "number",
                "description": "",
            },
            ...
        },
        "required": ["a", "b"]
    }
}
```

Agent Capabilities



Agent capabilities

Capabilities are reusable behaviors that can be added to agents. They can be a combinations of:

- **Toolsets** - builtin tools or custom tools
- **Lifecycle hooks** - intercept agent events
- **Instructions** - dynamically add additional instructions
- **Model settings** - dynamically adjust the model settings

```python
from pydantic_ai.capabilities import WebSearch

agent = Agent(
    model=model,
    instructions="You are a Python coding assistant.",
    capabilities=[
        WebSearch(),
    ],
)
```

Custom capabilities

appliedAI institute for europe

You can build your own custom capabilities by subclassing **`AbstractCapability`** and overriding specific methods.

```python
from dataclasses import dataclass
from typing import Any

from pydantic_ai.capabilities import AbstractCapability

@dataclass
class MyCapability(AbstractCapability[Any]):

    # Override the methods that you need
```

Capabilities that provide tools

The agent has tool calling functionality. We provide the agent with tools by implementing a capability that overrides the **`get_toolset()`** method.

```python
from dataclasses import dataclass
from typing import Any

from pydantic_ai.capabilities import AbstractCapability
from pydantic_ai.toolsets import FunctionToolset

@dataclass
class Calculator(AbstractCapability[Any]):
    def get_toolset(self) -> FunctionToolset:
        toolset = FunctionToolset()

        # add tools..

        return toolset
```

Function tools

Function tools are normal Python functions that we add to the toolset using the **`add_function()`** method.

```python
def add(a: float, b: float) -> float:
    """Add a to b."""
    return a + b

def sub(a: float, b: float) -> float:
    """Subtract b from a."""
    return a - b
```

```python
class Calculator(AbstractCapability[Any]):
    def get_toolset(self) -> FunctionToolset:
        toolset = FunctionToolset()

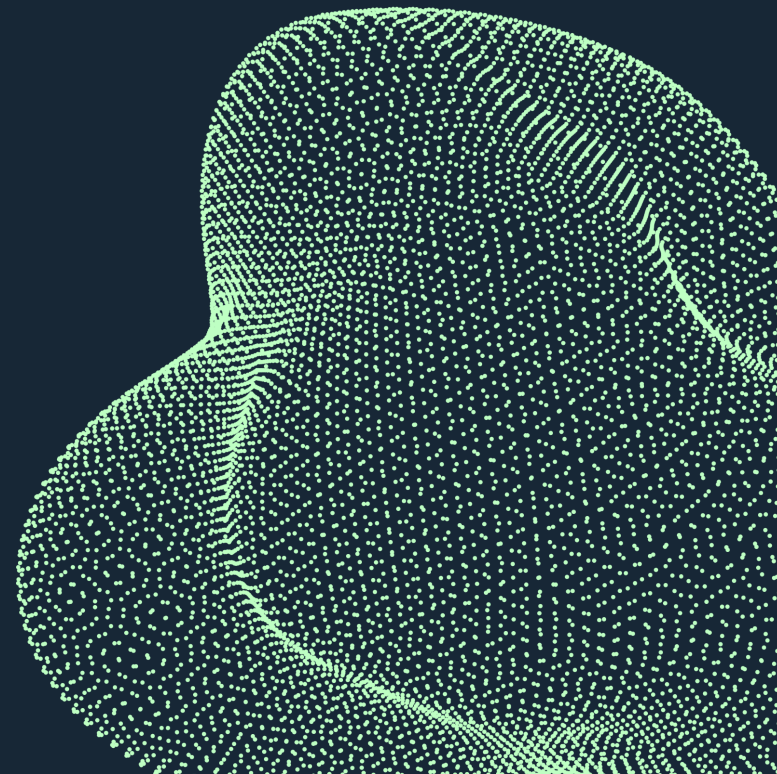
        toolset.add_function(add)
        toolset.add_function(sub)

        return toolset
```

appliedAI institute for europe

Exercise 🎒

Adding tool calling



Exercise 3: Adding tool calling

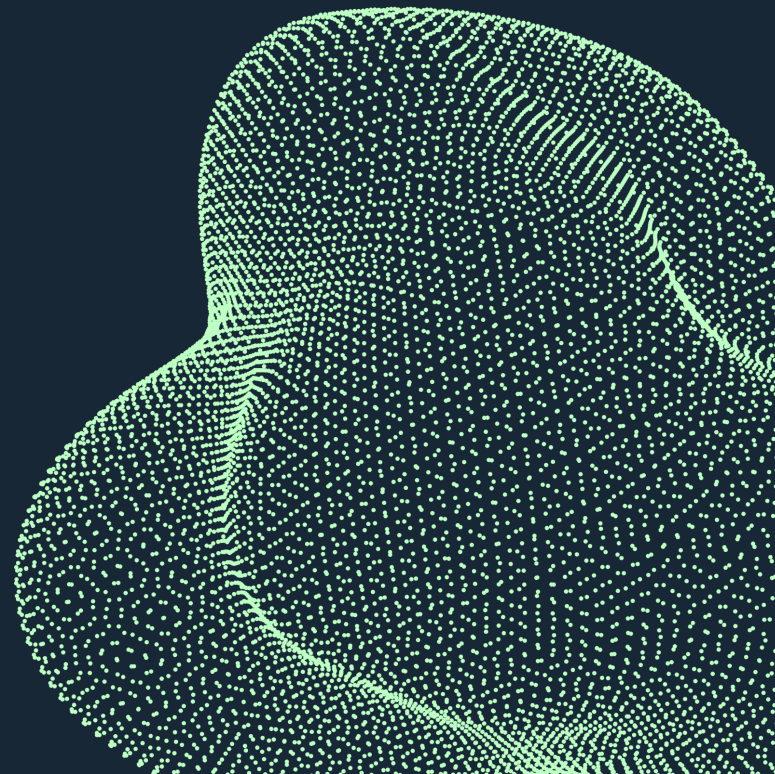
Goal: Give the agent basic file system capabilities through tools.

1. Implement the tool functions: **`read_file`** and **`write_file`**
2. Create the **`FileOperations`** capability.
3. Attach the capability to the agent.

⚠️ **Warning:** you are giving the agent access to your file system. Sandbox the file operations!

```
_path_sandbox(path).read_text()
_path_sandbox(path).write_text(content)
```

Run context



Run context
Agent dependencies

`RunContext` dependencies allow to **inject external state or services** into your functions/tools at runtime (e.g., API keys, database client, current user, etc.)

```python
from dataclasses import dataclass

@dataclass
class MyDeps:
    name: str
```

```python
from pydantic_ai import Agent

from .tools import greet

agent = Agent[MyDeps](
    ...
    tools=[greet]
    deps_type=MyDeps,
)

deps = MyDeps("Jan Willem")

await agent.run("Write me a greeting", deps=deps)
```

Run context

Accessing dependencies and run information

Tools can access the dependencies via **`ctx.deps`**:

```python
from pydantic_ai import RunContext

def greet(ctx: RunContext[MyDeps]) -> str:
    return f"Hello, {ctx.deps.name}"
```

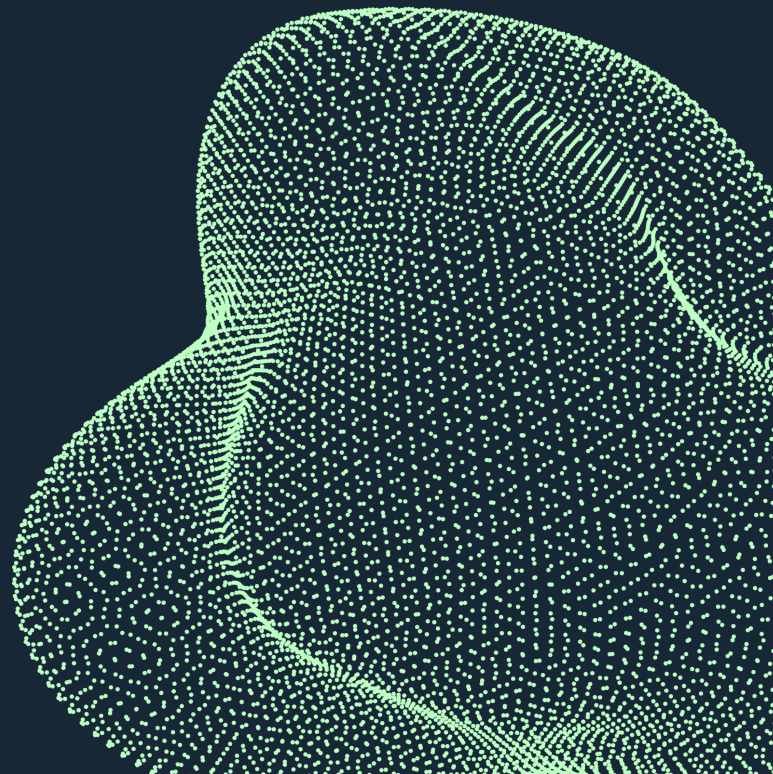
Note: The run context parameter is removed when sending the tool definition to the model.

Besides the dependencies, you can request information about the current run:

- **`model`**: The model used in this run
- **`usage`**: LLM usage for this run
- **`prompt`**: The original user prompt passed to the run
- **`messages`**: The messages exchanged so far
- **`tool_name`**: Name of the tool being called

appliedAI
institute
for europe

Hooks



Lifecycle hooks

Hooks allow to **intercept** and **modify** model requests, tool calls, and the overall run. Pydantic AI processes many (internal) events, and hooks allow us to tap into those event.

- **Run hooks**: overall run event
- **Node hooks**: internal agent graph node events
- **Model request hooks**: events related to requests made to the model
- **Tool hooks**: tool validation and execution events

- **`before_*`**: fires before the action (you may modify inputs)
- **`after_*`**: fires after the action is successful (you may modify the outputs)
- **`wrap_*`**: receive a callable and whether and how to call it
- **`on_*_error`**: fires when the actions fails

Capabilities that hook into the lifecycle

We hook into the agent lifecycle by creating a capability that overrides one, or more, hook-related methods.

```python
from dataclasses import dataclass
from typing import Any

from pydantic_ai import RunContext
from pydantic_ai.capabilities import AbstractCapability

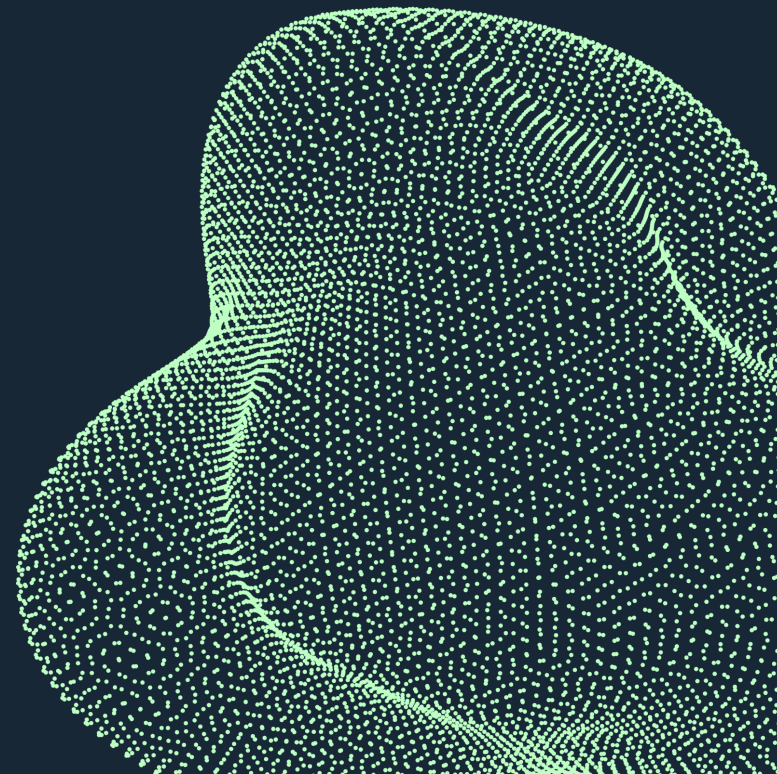
@dataclass
class Summary(AbstractCapability[Any]):

    def before_run(ctx: RunContext[Any]) -> None:
        summary = summarize(ctx.prompt)
        ctx.deps.console.rule(summary)
```

appliedAI institute for europe

Exercise 🎒

Adding visibility with hooks



Exercise 4: Adding visibility with hooks

Goal: Log file operation events to the console.

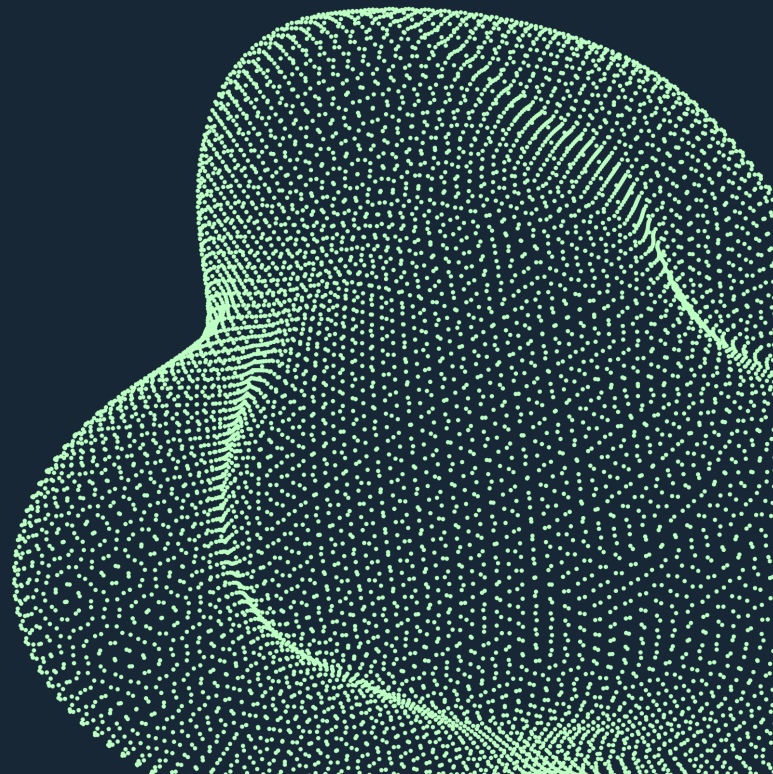
1. Set the **`deps_type`** to **`AgentDeps`** and pass the dependencies instance in the **`run()`** method.

2. Implement the **`before_tool_execute()`** hook.

3. Attach the capability to the agent.

```
>> Refactor the CSV parser ...
[14:42:12] Searching: **/*.py              file_operations.py:163
[14:42:17] Reading: csv_parser.py          file_operations.py:165
[14:42:04] Editing: csv_parser.py          file_operations.py:167
```

Model settings



Model settings

So far, we have been using the default model settings. Depending on your agent, or on the specific request send to the agent, you may want to adjust your model settings.

- **`max_tokens`**: the maximum number of tokens to generate before stopping
- **`temperature`**: the amount of randomness in the response
- **`thinking`**: enable/disable reasoning, or configure the reasoning effort

```python
from pydantic_ai import ModelSettings
from pydantic_ai.models.openai import OpenAIChatModel

settings = ModelSettings(
    thinking="high",
)

model = OpenAIChatModel(
    model_name="gpt-5.1-codex-mini",
    settings=settings,
)
```

Model settings

Capability that provides model settings

appliedAI institute for europe

As an alternative to passing **`settings`** when instantiating the agent, you can create a capability that provides the model settings.

```python
class ThinkHarder(AbstractCapability[Any]):
    def get_model_settings(self) -> Callable[[RunContext[Any]], ModelSettings]:

        def _set_reasoning_high(ctx: RunContext[Any]) -> ModelSettings:
            return ModelSettings(
                thinking="high"
            )

        return _set_reasoning_high
```

Direct vs Callable

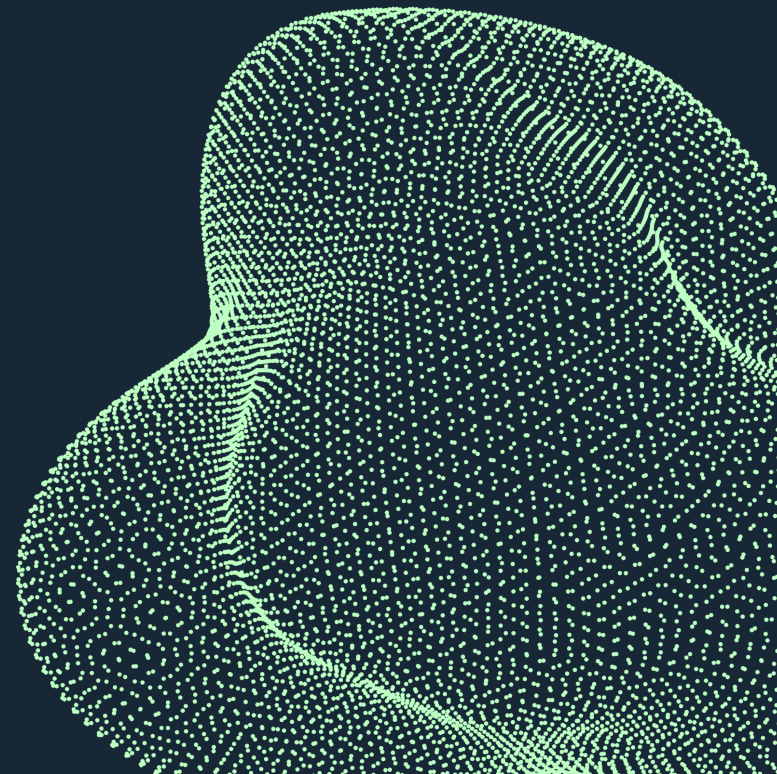
Methods like **`get_model_settings()`** allow you to return **`ModelSettings`** directly, or return a **`Callable`** (e.g., a Python function) that returns the settings.

- Return **`ModelSettings`** directly when the model settings won't change between runs.
- Use a **`Callable`** when settings depend on the run:
  - For example, you need to access attributes in the **`RunContext`**

appliedAI institute for europe

Exercise 🎒

Controlling reasoning effort



Exercise 5: Controlling reasoning effort

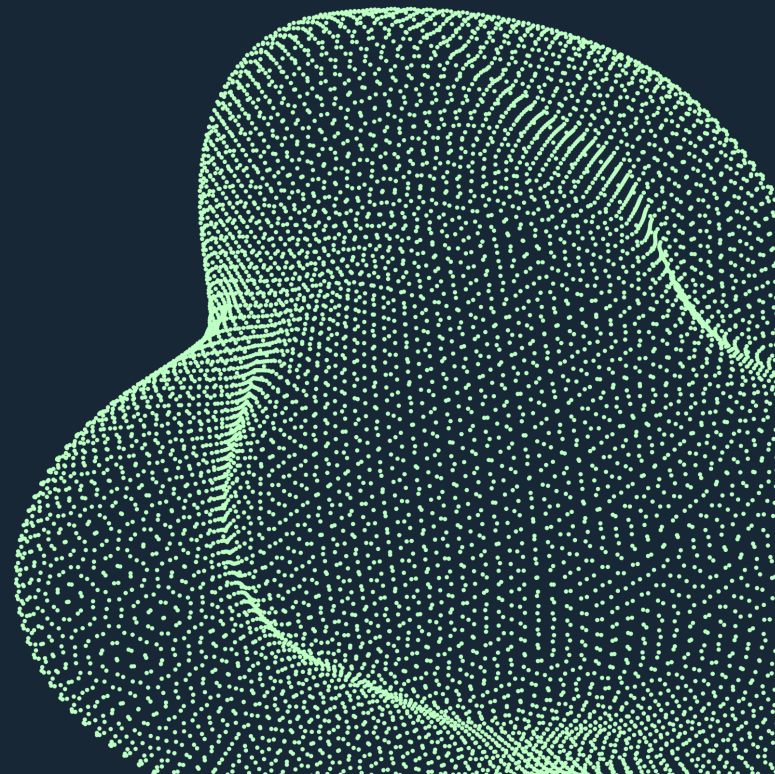
Goal: Dynamically adjust the reasoning effort based on the user prompt. For example, by scanning the prompt for `"@high"` or `"@low"`.

1. Create a **`ReasoningEffort`** capability and implement the **`get_model_settings()`** method.

2. Scan **`ctx.prompt`** and set the reasoning effort field accordingly.

3. Attach the **`ReasoningEffort`** capability to the agent.

Dynamic system instructions



Dynamic system instructions

The system instructions passed via **`instructions`** are fixed when creating the agent instance. You can add additional instructions, which are dynamically added on each run (current date, current user, etc.).

```
def _current_date(ctx: RunContext[Any]) -> str:
    now = datetime.now()
    return f"The current date and time is {now.isoformat()}"

agent = Agent(
    model=model,
    instructions="You are a helpful assistant"
)

agent.instructions(_current_date)
```

Dynamic system instructions
Capabilities that provide instructions

Alternatively, you can create a capability that provides instructions. This is especially useful when the capability provides both a toolset, and instructions to use the tools in the toolset.

```python
@dataclass
class CurrentDateTime(AbstractCapability[Any]):
    def get_instructions(self) -> Callable[[RunContext[Any]], str]:
        def _current_date(ctx: RunContext[Any]) -> str:
            now = datetime.now()
            return f"The current date and time is {now.isoformat()}"

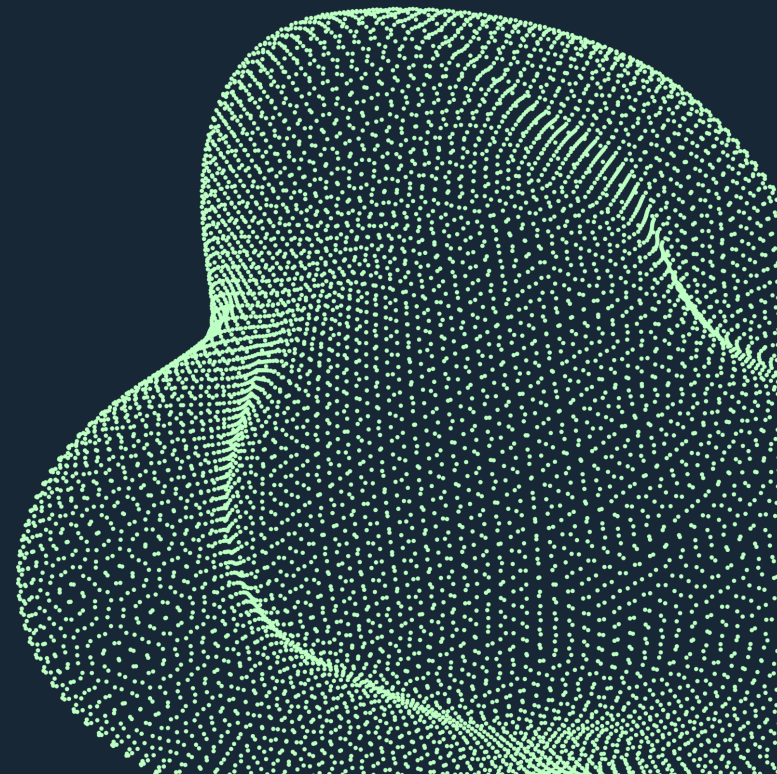
        return _current_date
```

Note: similar to **`get_model_settings()`**, **`get_instructions()`** allows you to return a string directly. Use this to add additional "default" instructions that won't change in-between runs.

appliedAI institute for europe

Exercise 🎒

Making the agent extensible with skills



Exercise 6: Making the agent extensible with skills

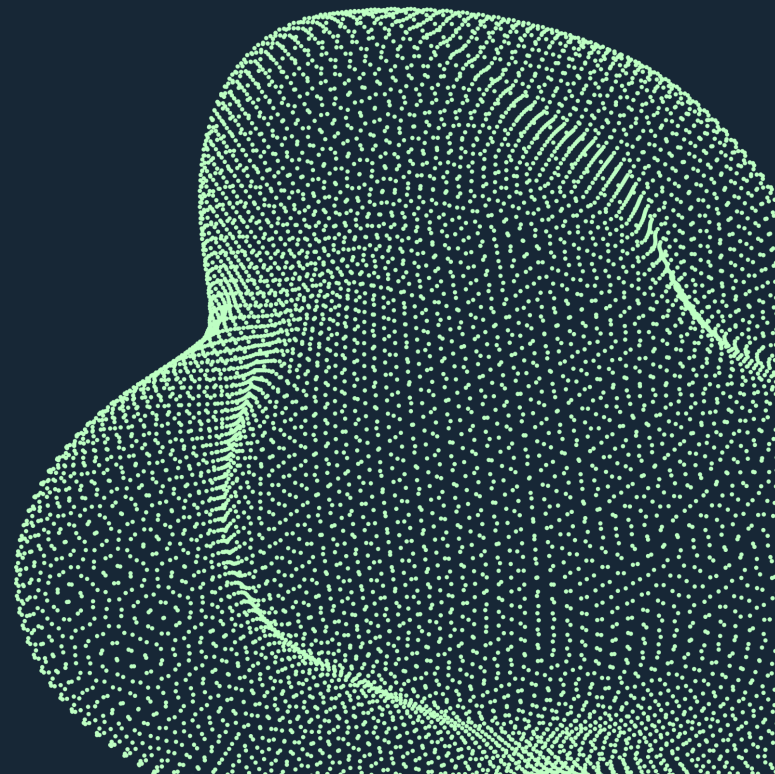
Goal: Complete the skills implementation in `skills.py`. Provide the model with a listing of the available skills, so that the model knows which skills it can load with the **`load_skill()`** tool.

1. Extends the **`Skills`** capability by implementing the **`get_instructions()`** method.

2. Attach the **`Skills()`** capability to the agent.

appliedAI institute for europe

Structured output



Structured output

appliedAI institute for europe

By default, the output type of **`Agent`** is **`str`**. You can enforce the model output via Pydantic model specifications.

```python
from pydantic import BaseModel
from pydantic_ai import Agent

class Person(BaseModel):
    name: str
    age: int

agent = Agent[Person](
    model=model,
    instructions="You are a helpful assistant.",
    output_type=Person,
)
```

```python
result = await agent.run(
    "Generate a random person with name and age"
)

print(result.data.name)  # Alice
print(result.data.age)   # 30
```

Structured output
Plan and execute agents

Structured output is especially helpful when building more complex agent architectures. For example, one agent creating a plan, another agent implementing this plan.

```python
from pydantic import BaseModel
from pydantic_ai import Agent

class Plan(BaseModel):
    goal: str
    steps: list[str]

plan_agent = Agent[Plan](
    model=model,
    instructions=(
        "You are a Python coding assistant.\n"
        "Your task is to make an implementation plan."
    ),
    output_type=Plan,
)
```

```python
result = await plan_agent.run(
    "Refactor this module into..."
)

goal = result.data.goal
steps = result.data.steps
```

Structured output
Plan and execute agents

Structured output is especially helpful when building more complex agent architectures. For example, one agent creating a plan, another agent implementing this plan.

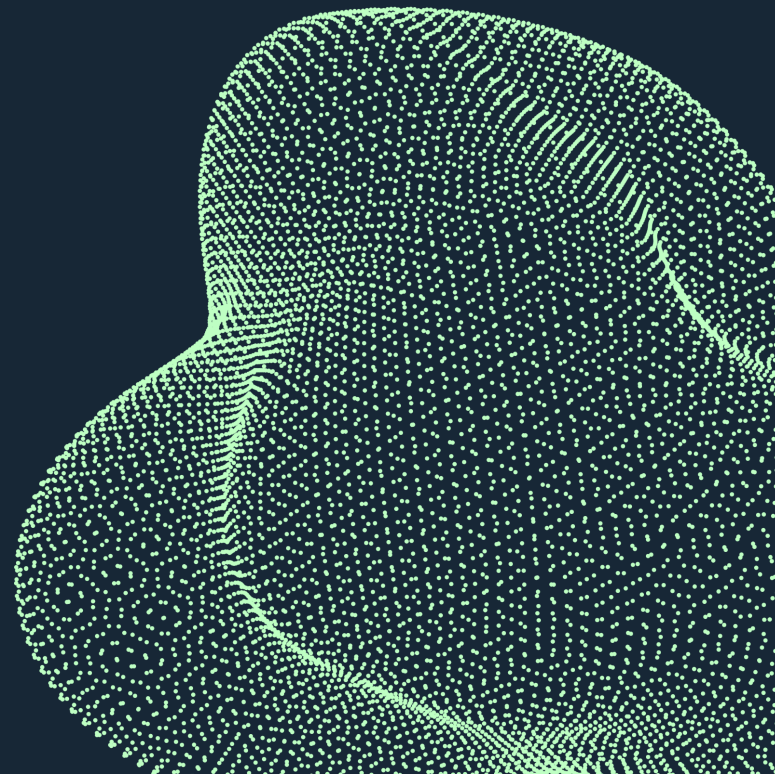
```python
from pydantic_ai import Agent

execute_agent = Agent(
    model=model,
    instructions=(
        "You are a Python coding assistant.\n"
        "Implement the solution step by step.\n"
        f"The overall goal is '{goal}'"
    ),
    capabilities=[
        FileOperations(),
        Skills(),
    ],
)
```

```python
goal = result.data.goal
steps = result.data.steps

for step in steps:
    await execute_agent.run(
        f"Implement step: {step}"
    )
```

Where to go from here...



A couple things we did not talk about..

- **Observability/tracing:**
  Get a full trace of the agent execution (including reasoning, outputs, and tool calls) for monitoring and debugging agents.

  Tip: Use MLflow for an easy tracing setup:

```
import mlflow

mlflow.set_experiment("my-second-agent")
mlflow.pydantic_ai.autolog()
```

- **Evaluations:**
  Structurally test how your agent is performing and identify where it makes mistakes or show undesired behavior.
- **Chat interfaces:**
  Pydantic AI supports the Vercel AI SDK and AG-UI
- **Agent teams/swarms/...:**
  Pydantic AI implements the A2A protocol.

But...

appliedAI institute for europe

Keep it simple!

Levels of autonomy: augmentation, automation, and autonomous operations

Deployment patterns across organizational layers

appliedAI institute for europe

Individual augmentation

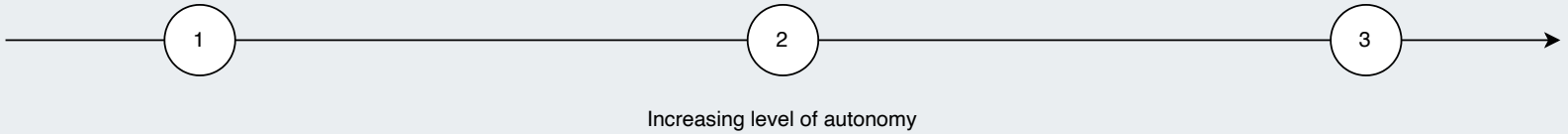
- Personal assistants, task automation, decision support
- User-scoped permissions, personal context management

Process automation

- Agent handles a (sub)process end-to-end
- Integration with existing systems and workflows
- System-level permissions

Autonomous operations

- Agent handles ongoing processes with minimal intervention
- Robust error handling, escalation protocols, logging



Note: You do not aim for increasing autonomy. In practice, you pick the lowest level that meets the use case.

Levels of human involvement: in-the-loop, on-the-loop, over-the-loop

Deployment patterns across organizational layers

The level of human involvement and oversight depends on risk, reversibility, and the required accountability for a use case:

Human-in-the-loop

Human approval is required before critical actions. AI produces suggestions, human validates, edits, or rejects. Emphasis on oversight and accountability.

high-risk
and regulated environments

Human-on-the-loop

AI executes autonomously within boundaries. Human monitors and can intervene. Alerts and logging is in place. Focus on supervision.

medium-risk
operational workflows

Human-out-the-loop

AI acts autonomously end-to-end. No real-time human supervision. Relies on validation, testing, and safeguards. Periodic human audits.

low-risk
or reversible actions

Summary

In this masterclass, we build a simple functioning coding assistant:

- Introduction to AI Agents
- Building agentic systems using Pydantic AI
- Extending the agent with capabilities:
  - Tool calling
  - Hook into the agent lifecycle
  - Model settings
  - Dynamic system prompts
- Structured output
- Discussed where to go from here

Questions?

appliedAI institute for europe

```
agent = Agent(
    model=model,
    instructions="You are a Q&A agent."
)

result = agent.run("I got a question! ...")
print(result.output)  # That is a great question! ...
```